CSA1708-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 5: write the python program for missionaries cannibal problem

Aim:

To write a Python program to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm.

Algorithm:

1. Start the program.
2. Initialize three missionaries and three cannibals on the left bank, with the boat on the left bank.
3. Define the possible boat movements: one missionary, two missionaries, one cannibal, two cannibals, or one missionary and one cannibal.
4. Use a queue to explore possible states using BFS.
5. Check that no bank has more cannibals than missionaries when missionaries are present.
6. Reject invalid states and avoid revisiting states.
7. Continue until all missionaries and cannibals reach the right bank.
8. Print the solution path.
9. Stop the program.



```
py4.py - C:/Users/navit/py4.py (3.12.10)
File  Edit  Format  Run  Options  Window  Help
from collections import deque

def missionaries_cannibals():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    moves = [(1, 0), (2, 0), (0, 1),
             (0, 2), (1, 1)]

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue
        visited.add(state)

        if state == goal:
            print("Solution Found:\n")
            for step, s in enumerate(path + [state]):
                m, c, boat = s
                side = "Left" if boat == 1 else "Right"
                print(f"Step {step}: Missionaries={m}, "
                      f"Cannibals={c}, Boat={side}")
            return

        m, c, boat = state

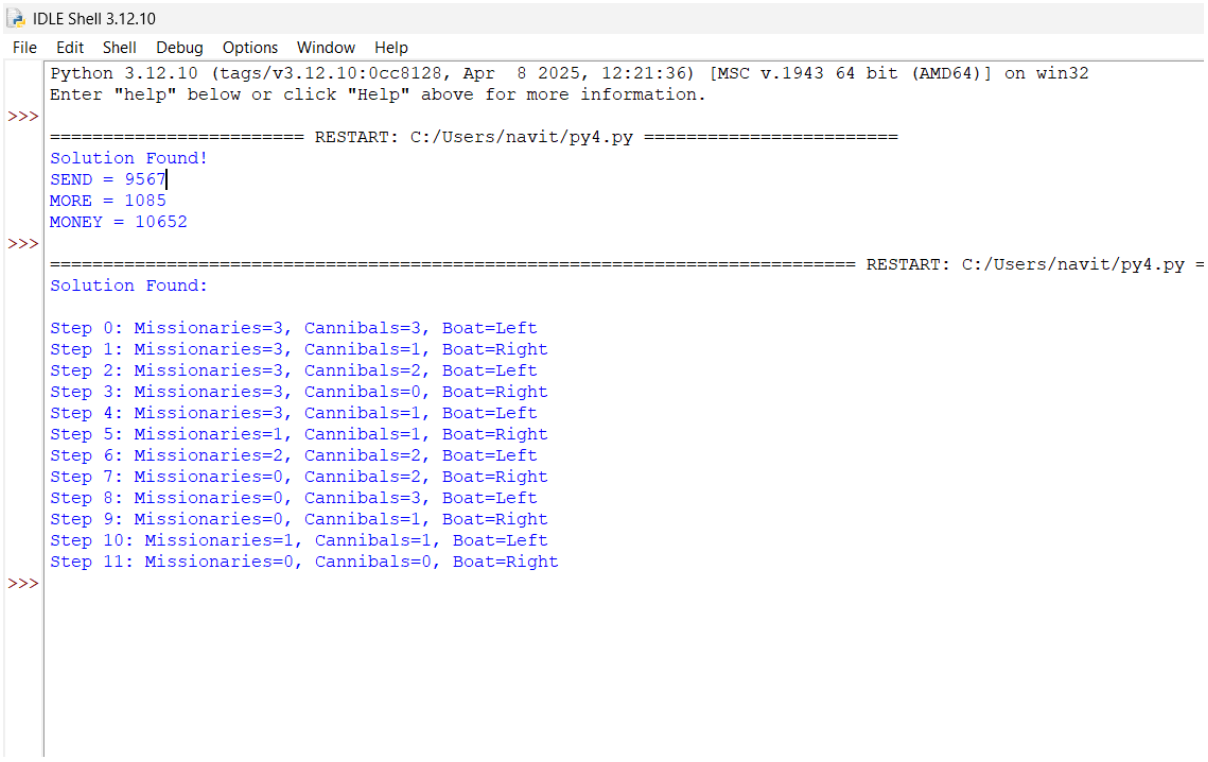
        for dm, dc in moves:
            if boat == 1:
                nm, nc = m - dm, c - dc
                nb = 0
            else:
                nm, nc = m + dm, c + dc
                nb = 1

            if not (0 <= nm <= 3 and 0 <= nc <= 3):
                continue

            rm, rc = 3 - nm, 3 - nc

            if (nm > 0 and nm < nc) or (rm > 0 and rm < rc):
                continue

            new state = (nm, nc, nb)
```



```
IDLE Shell 3.12.10
File  Edit  Shell  Debug  Options  Window  Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/navit/py4.py ==========
Solution Found!
SEND = 9567
MORE = 1085
MONEY = 10652
>>>
========== RESTART: C:/Users/navit/py4.py =
Solution Found:

Step 0: Missionaries=3, Cannibals=3, Boat=Left
Step 1: Missionaries=3, Cannibals=1, Boat=Right
Step 2: Missionaries=3, Cannibals=2, Boat=Left
Step 3: Missionaries=3, Cannibals=0, Boat=Right
Step 4: Missionaries=3, Cannibals=1, Boat=Left
Step 5: Missionaries=1, Cannibals=1, Boat=Right
Step 6: Missionaries=2, Cannibals=2, Boat=Left
Step 7: Missionaries=0, Cannibals=2, Boat=Right
Step 8: Missionaries=0, Cannibals=3, Boat=Left
Step 9: Missionaries=0, Cannibals=1, Boat=Right
Step 10: Missionaries=1, Cannibals=1, Boat=Left
Step 11: Missionaries=0, Cannibals=0, Boat=Right
>>>
```

Result:

Thus, the Python program solves the Missionaries and Cannibals Problem using BFS while ensuring that missionaries are never outnumbered by cannibals on either bank when missionaries are present.